# Chapter 3 Selections

# The `boolean` Type and Operators

Often in a program you need to compare two values, such as whether i is greater than j. Java provides six comparison operators (also known as relational operators) that can be used to compare two values. The result of the comparison is a Boolean value: true or false.

```
boolean b = (1 > 2);
```

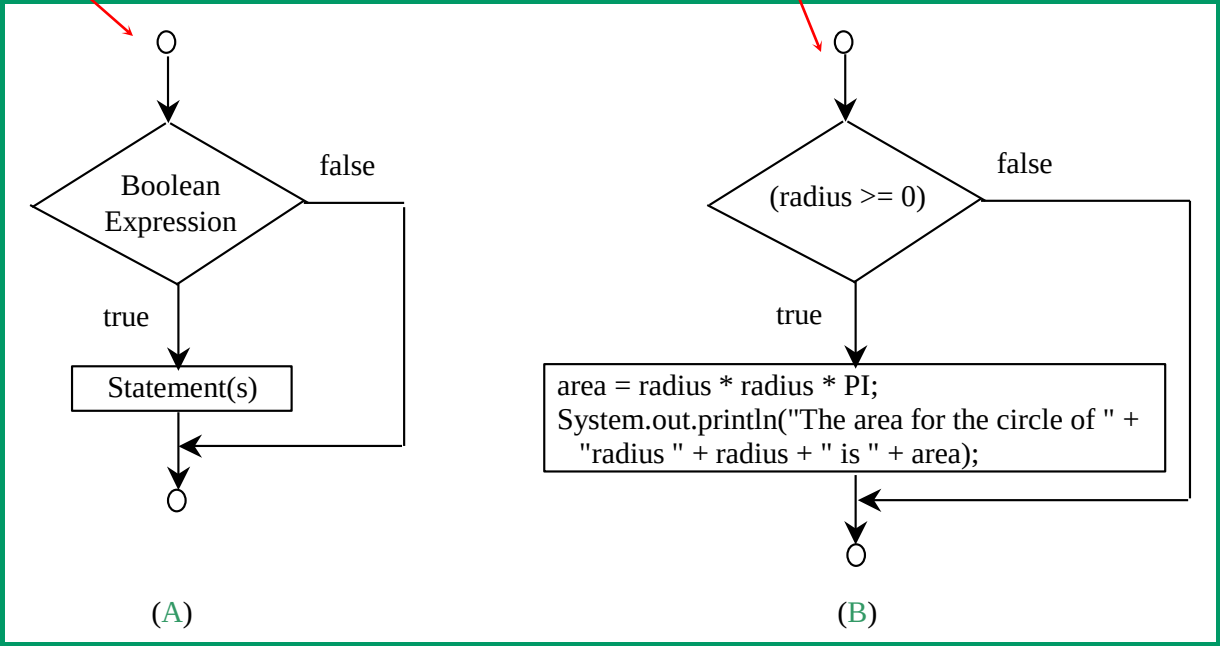
# Comparison Operators

| *Operator* | *Name* |
|---|---|
| < | less than |
| <= | less than or equal to |
| > | greater than |
| >= | greater than or equal to |
| == | equal to |
| != | not equal to |

# One-way `if` Statements

```
if (boolean-expression) {
  statement(s);
}
```

```
if (radius >= 0) {
  area = radius * radius * PI;
  System.out.println("The area for the circle of radius "
    + radius + " is " + area);
}
```



# Note

```
if i > 0 {
  System.out.println("i is positive");
}
```

(a) Wrong

```
if (i > 0) {
  System.out.println("i is positive");
}
```

(b) Correct

```
if (i > 0) {
  System.out.println("i is positive");
}
```

(a)

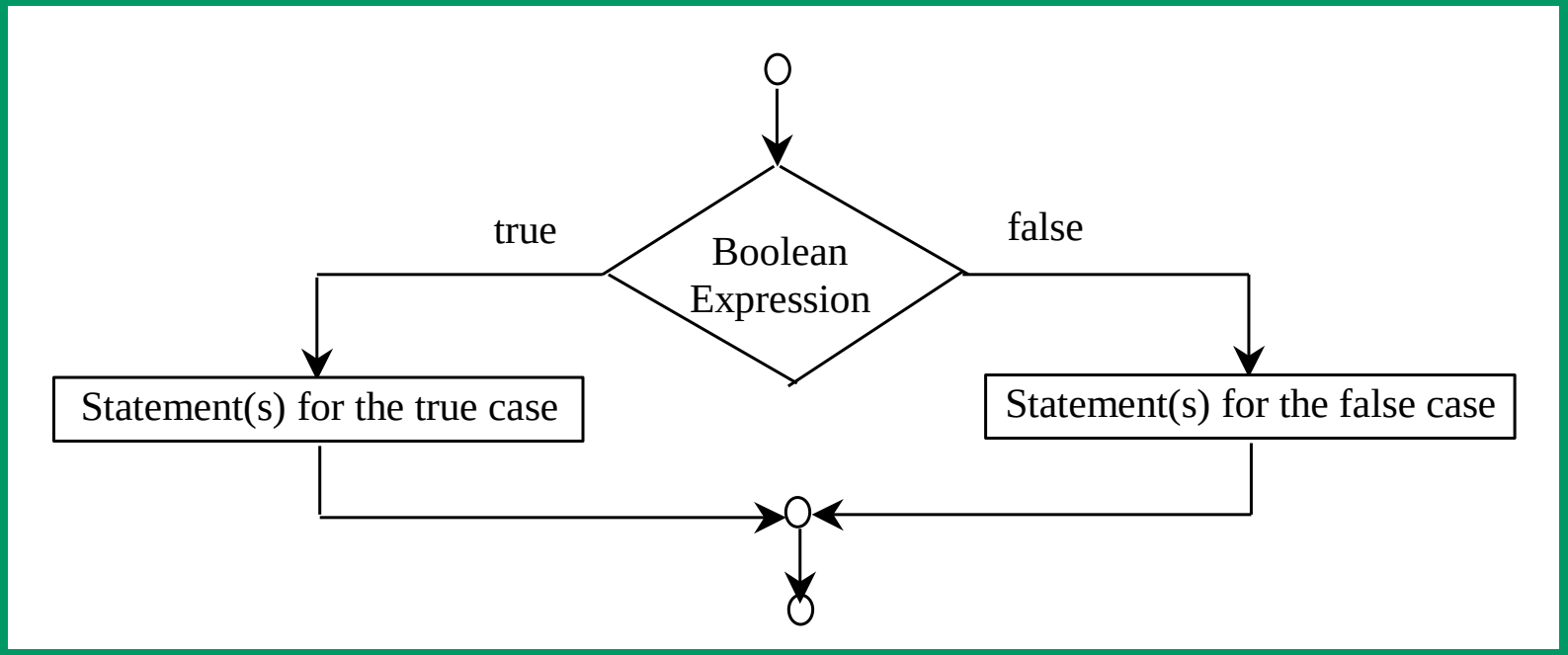
Equivalent

```
if (i > 0)
  System.out.println("i is positive");
```

(b)

# The Two-way `if` Statement

```
if (boolean-expression) {
  statement(s)-for-the-true-case;
}
else {
  statement(s)-for-the-false-case;
}
```



# `if...else` Example

```
if (radius >= 0) {
  area = radius * radius * 3.14159;

  System.out.println("The area for the "
    + "circle of radius " + radius +
    " is " + area);
}
else {
  System.out.println("Negative input");
}
```

# Multiple Alternative if Statements

```
if (score >= 90.0)
  grade = 'A';
else
  if (score >= 80.0)
    grade = 'B';
  else
    if (score >= 70.0)
      grade = 'C';
    else
      if (score >= 60.0)
        grade = 'D';
      else
        grade = 'F';
```

Equivalent

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

animation

# Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

animation

# Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

animation

# Trace if-else statement

Suppose score is 70.0

The condition is true

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

animation

# Trace if-else statement

Suppose score is 70.0

grade is C

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

animation

# Trace if-else statement

Suppose score is 70.0

Exit the if statement

```
if (score >= 90.0)
  grade = 'A';
else if (score >= 80.0)
  grade = 'B';
else if (score >= 70.0)
  grade = 'C';
else if (score >= 60.0)
  grade = 'D';
else
  grade = 'F';
```

# Common Errors

Adding a semicolon at the end of an <u>if</u> clause is a common mistake.

```
if (radius >= 0);
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

Wrong

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.

This error often occurs when you use the next-line block style.

# Problem: Body Mass Index

Body Mass Index (BMI) is a measure of health on weight. It can be calculated by taking your weight in kilograms and dividing by the square of your height in meters. The interpretation of BMI for people 16 years or older is as follows:

| BMI | Interpretation |
|---|---|
| below 16 | serious underweight |
| 16-18 | underweight |
| 18-24 | normal weight |
| 24-29 | overweight |
| 29-35 | seriously overweight |
| above 35 | gravely overweight |

ComputeBMI

Run

# Logical Operators

| *Operator* | *Name* |
|---|---|
| ! | not |
| && | and |
| \|\| | or |
| ^ | exclusive or |

# Truth Table for Operator !

| `p` | `!p` | Example (assume age = 24, gender = 'M') |
|---|---|---|
| `true` | `false` | !(age > 18) is false, because (age > 18) is true. |
| `false` | `true` | !(gender == 'F') is true, because (gender == 'F') is false. |

# Truth Table for Operator &&

| p1 | p2 | p1 && p2 | Example (assume age = 24, gender = 'F') |
|---|---|---|---|
| false | false | false | (age > 18) && (gender == 'F') is true, because (age > 18) and (gender == 'F') are both true. |
| false | true | false | |
| true | false | false | (age > 18) && (gender != 'F') is false, because (gender != 'F') is false. |
| true | true | true | |

# Truth Table for Operator ||

| p1 | p2 | p1 \|\| p2 | Example (assume age = 24, gender = 'F') |
|---|---|---|---|
| false | false | false | (age > 34) \|\| (gender == 'F') is true, because (gender == 'F') is true. |
| false | true | true | |
| true | false | true | (age > 34) \|\| (gender == 'M') is false, because (age > 34) and (gender == 'M') are both false. |
| true | true | true | |

# Examples

Here is a program that checks whether a number is divisible by 2 and 3, whether a number is divisible by 2 or 3, and whether a number is divisible by 2 or 3 but not both:

TestBooleanOperators

Run

# Truth Table for Operator !

| `p` | `!p` |
|---|---|
| `true` | `false` |
| `false` | `true` |

| Example |
|---|
| !(1 > 2) is true, because (1 > 2) is false. |
| !(1 > 0) is false, because (1 > 0) is true. |

# Truth Table for Operator &&

| p1 | p2 | p1 && p2 |
|---|---|---|
| false | false | false |
| false | true | false |
| true | false | false |
| true | true | true |

| Example |
|---|
| (3 > 2) && (5 >= 5) is true, because (3 > 2) and (5 >= 5) are both true. |
| (3 > 2) && (5 > 5) is false, because (5 > 5) is false. |

# Truth Table for Operator ||

| p1 | p2 | p1 \|\| p2 |
|---|---|---|
| false | false | false |
| false | true | true |
| true | false | true |
| true | true | true |

| Example |
|---|
| (2 > 3) \|\| (5 > 5) is false, because (2 > 3) and (5 > 5) are both false. |
| (3 > 2) \|\| (5 > 5) is true, because (3 > 2) is true. |

# Truth Table for Operator ^

| p1 | p2 | p1 ^ p2 | Example (assume age = 24, gender = 'F') |
|---|---|---|---|
| false | false | false | (age > 34) ^ (gender == 'F') is true, because (age > 34) is false but (gender == 'F') is true. |
| false | true | true | |
| true | false | true | (age > 34) \|\| (gender == 'M') is false, because (age > 34) and (gender == 'M') are both false. |
| true | true | false | |

# Problem: Lottery

Write a program that randomly generates a lottery of a two-digit number, prompts the user to enter a two-digit number, and determines whether the user wins according to the following rule:

- If the user input matches the lottery in exact order, the award is $10,000.
- If the user input matches the lottery, the award is $3,000.
- If one digit in the user input matches a digit in the lottery, the award is $1,000.

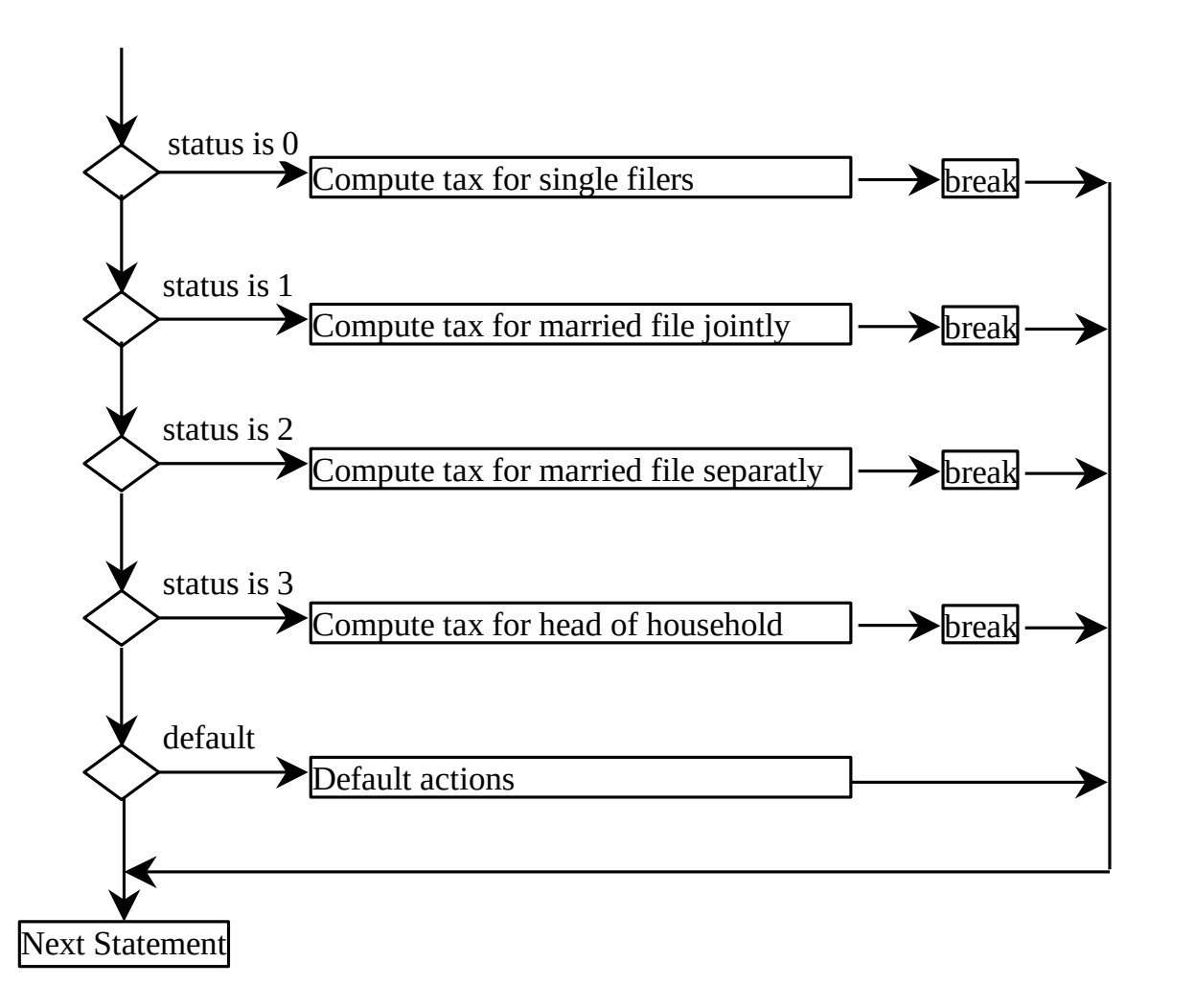
Lottery

Run

# `switch` Statements

```
switch (status) {
  case 0:  compute taxes for single filers;
           break;
  case 1:  compute taxes for married file jointly;
           break;
  case 2:  compute taxes for married file separately;
           break;
  case 3:  compute taxes for head of household;
           break;
  default: System.out.println("Errors: invalid status");
           System.exit(0);
}
```

# `switch` Statement Flow Chart



# `switch` Statement Rules

The switch-expression must yield a value of char, byte, short, or int type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as 1 + x.

```
switch (switch-expression) {
  case value1:  statement(s)1;
          break;
  case value2: statement(s)2;
          break;
  …
  case valueN: statement(s)N;
          break;
  default: statement(s)-for-default;
}
```

# `switch` Statement Rules

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

```
switch (switch-expression) {
  case value1:  statement(s)1;
        break;
  case value2: statement(s)2;
        break;
  …
  case valueN: statement(s)N;
        break;
  default: statement(s)-for-default;
}
```

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

The case statements are executed in sequential order, but the order of the cases (including the default case) does not matter. However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

animation

# Trace switch statement

Suppose ch is 'a':

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

ch is 'a':

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute this line

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute this line

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute this line

```
switch (ch) {
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute next statement

```
switch (ch)
  case 'a': System.out.println(ch);
  case 'b': System.out.println(ch);
  case 'c': System.out.println(ch);
}
```

Next statement;

animation

# Trace switch statement

Suppose ch is 'a':

```
switch (ch) {
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

ch is 'a':

```
switch (ch) {
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute this line

```
switch (ch) {
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute this line

```
switch (ch) {
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

animation

# Trace switch statement

Execute next statement

```
switch (ch)
  case 'a': System.out.println(ch);
            break;
  case 'b': System.out.println(ch);
            break;
  case 'c': System.out.println(ch);
}
```

Next statement;

# Conditional Operator

```
if (x > 0)
  y = 1
else
  y = -1;
```

is equivalent to

```
y = (x > 0) ? 1 : -1;
```

(boolean-expression) ? expression1 : expression2

Ternary operator

# Conditional Operator

```
if (num % 2 == 0)
  System.out.println(num + “is even”);
else
  System.out.println(num + “is odd”);


System.out.println(
  (num % 2 == 0)? num + “is even” :
  num + “is odd”);
```

# Conditional Operator, cont.

```
(boolean-expression) ? exp1 : exp2
```

# Formatting Output

Use the printf statement.

```
System.out.printf(format, items);
```

Where format is a string that may consist of substrings and format specifiers. A format specifier specifies how an item should be displayed. An item may be a numeric value, character, boolean value, or a string. Each specifier begins with a percent sign.

# Frequently-Used Specifiers

| Specifier | Output | Example |
|---|---|---|
| %b | a boolean value | true or false |
| %c | a character | 'a' |
| %d | a decimal integer | 200 |
| %f | a floating-point number | 45.460000 |
| %e | a number in standard scientific notation | 4.556000e+01 |
| %s | a string | "Java is cool" |

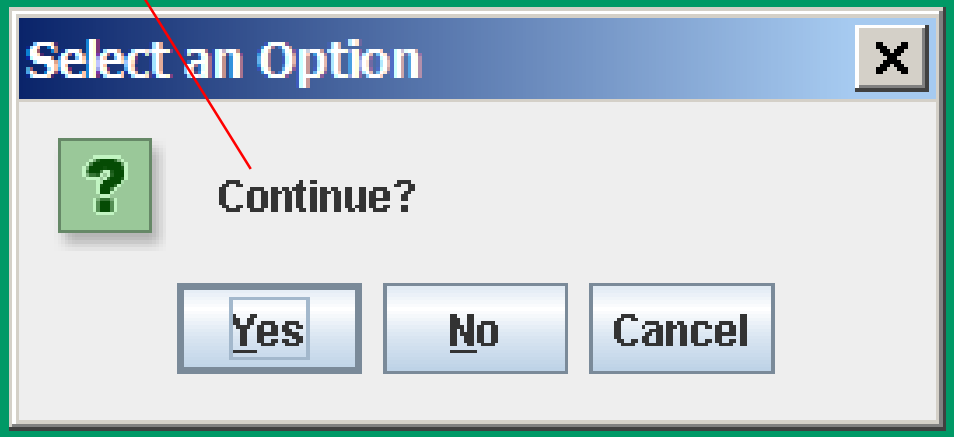
```
int count = 5;
double amount = 45.56;
System.out.printf("count is %d and amount is %f", count, amount);
```

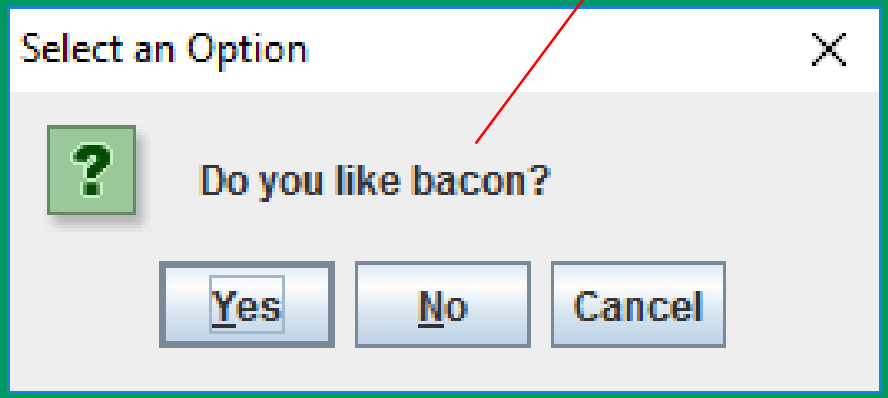
items

```
display            count is 5 and amount is 45.560000
```

# (GUI) Confirmation Dialogs

```
int option = JOptionPane.showConfirmDialog
    (null, "Continue");
```



# (GUI) Confirmation Dialogs

```
import javax.swing.JOptionPane;

public class ConfirmDialog1 {

public static void main(String[] args) {
   int input = JOptionPane.showConfirmDialog(null, "Do you like bacon?");
   // 0=yes, 1=no, 2=cancel
   System.out.println(input);
   }
}
```